<< GENETIC ALGORITHMS >>

Genetic Algorithms are intelligent search techniques maintaining a population of candidate solutions for a given problem and search the solution space by applying variation operators.

Nature	Genetic Algorithm
Environment	Optimization Problem
Individuals	Feasible Solution
Individual's Adaptation	Solution Quality
Selection, Recombination, and Reproduction	Variation Operators

What is Genetic Algorithms (G.A.)?

1. **G.A.s** refer to a family of computational models inspired by Darwin's theory of biological evolution – Survival of the Fittest.
2. The idea is one of Natural Selection organizing principle for optimizing individuals and
3. The idea is one of Natural Selection organizing principle for optimizing individuals and populations of individuals.
4. GAs mimic Natural Selection to optimize more successfully.
5. Problems are solved by an evolutionary process resulting in a best (fittest) solution (survivor).

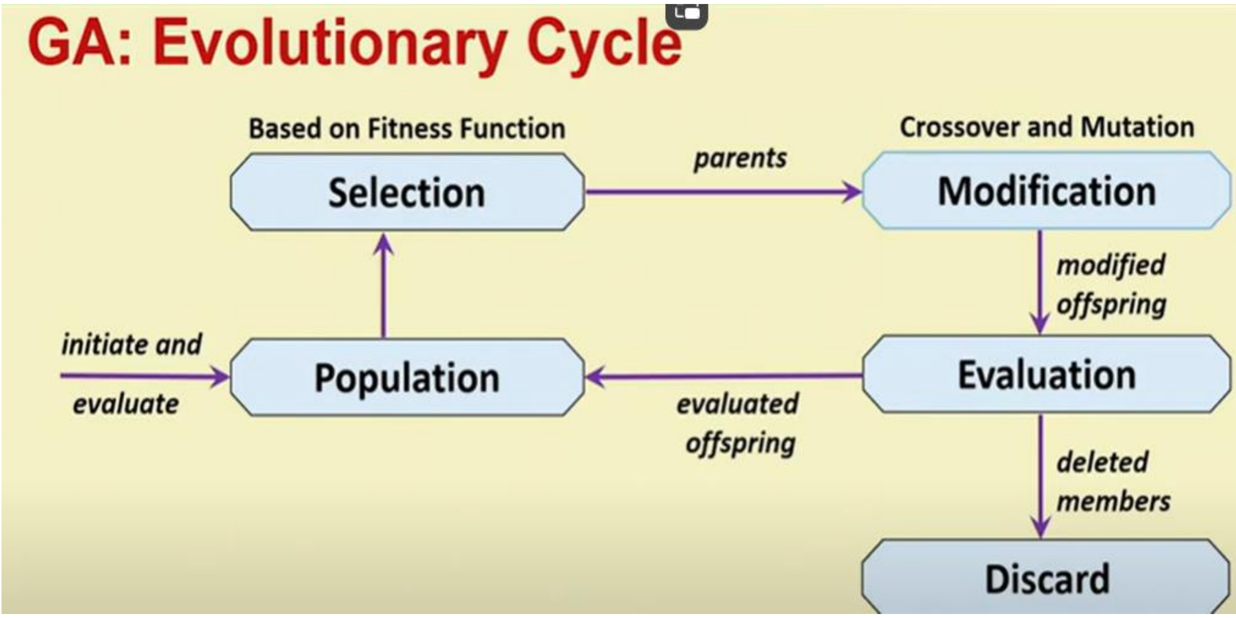
Genetic Algorithm vs Search Techniques

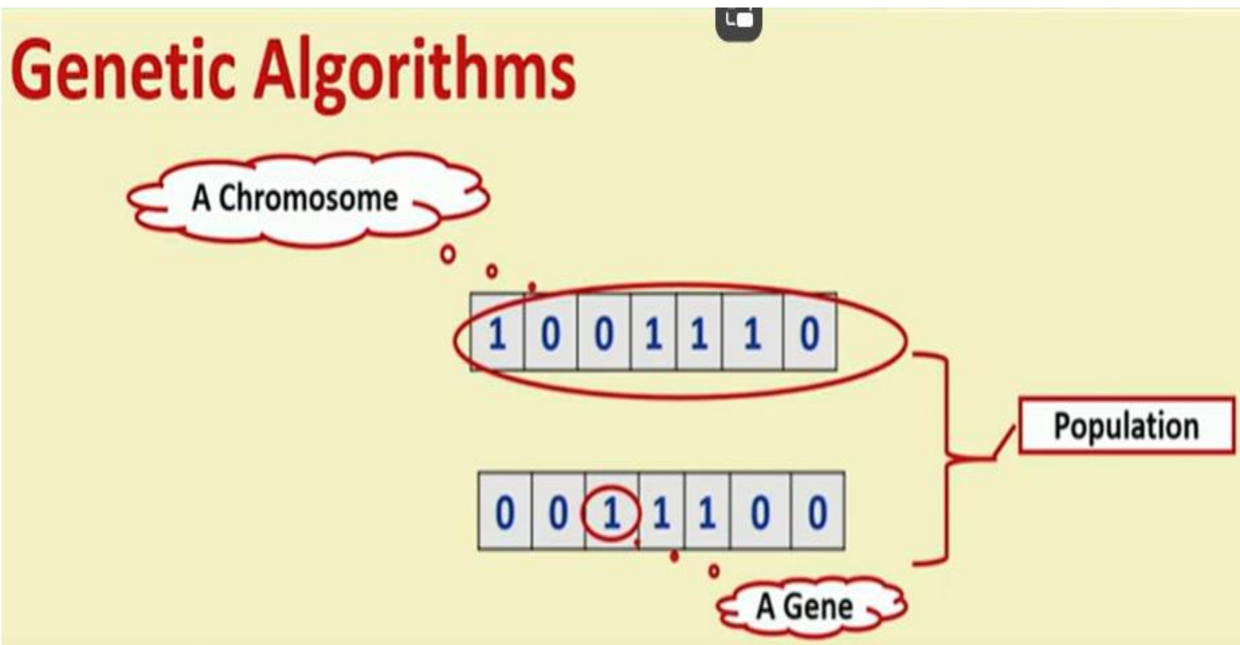
1. Inspired by "Natural evolution", GAs involve direct manipulation of the coding achieved by the crossover and mutation operators.
2. GAs begin their search from many points, not from a single point, contain population of feasible solutions to the problem.
3. GAs do not need auxiliary information like gradients at points. They search via sampling.
4. GAs search by stochastic operators, not by deterministic rules. They use random choice to guide highly exploitative search.

Genetic Algorithm Process

1. Encode potential solutions in terms of chromosome-like data structure.
2. Select parents on the basis of the fitness of the solutions to produce offspring for next generation, who contain the characteristics of both parents.
3. Employ recombination operators (selection, crossover and mutation) repeatedly to preserve the good portions of the strings.
4. Good portions of the strings usually lead to an optimal or near-optimal solution. The method is applied over a desired number of generations.

5. If well designed, population will converge faster.

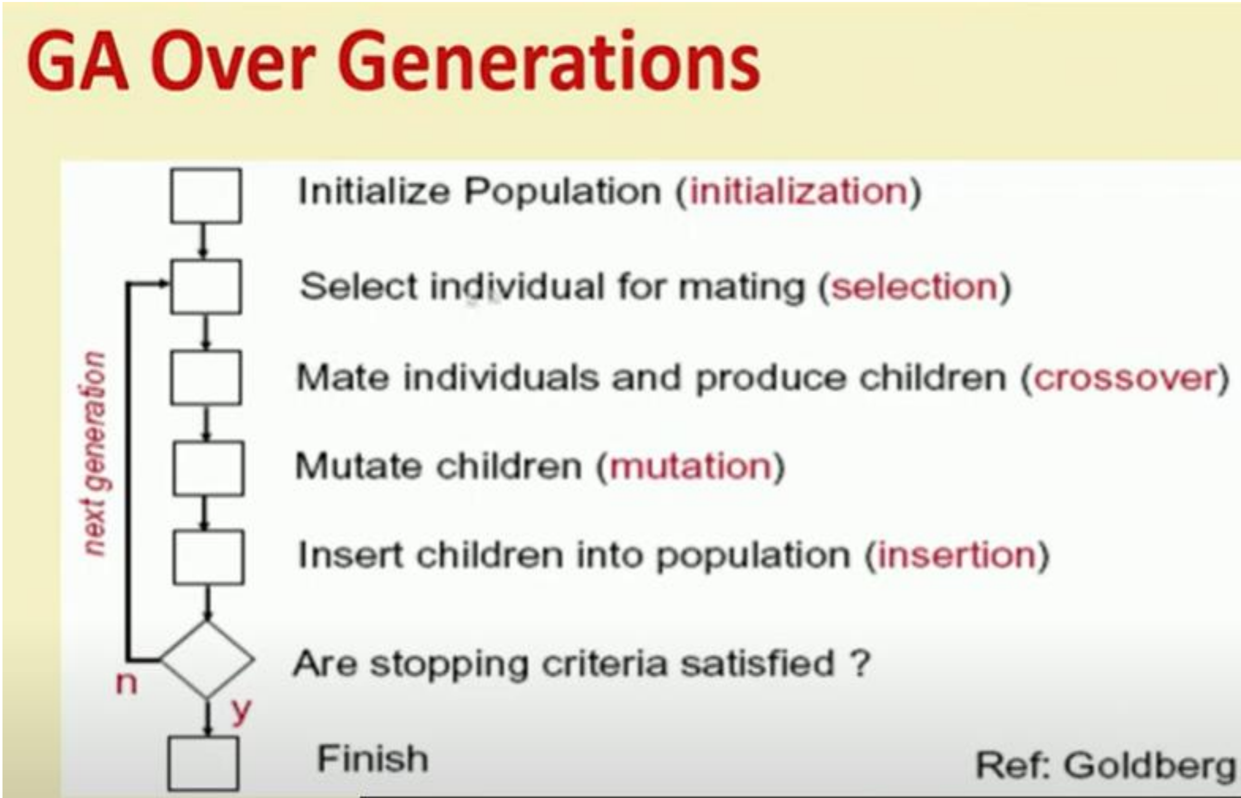




REMARK: (i) Binary Encoding uses 0's and 1's in a chromosome.

(ii) Each bit corresponds to a "gene".

(iii) The values for a given gene are "alleles".



Chromosome Encoding

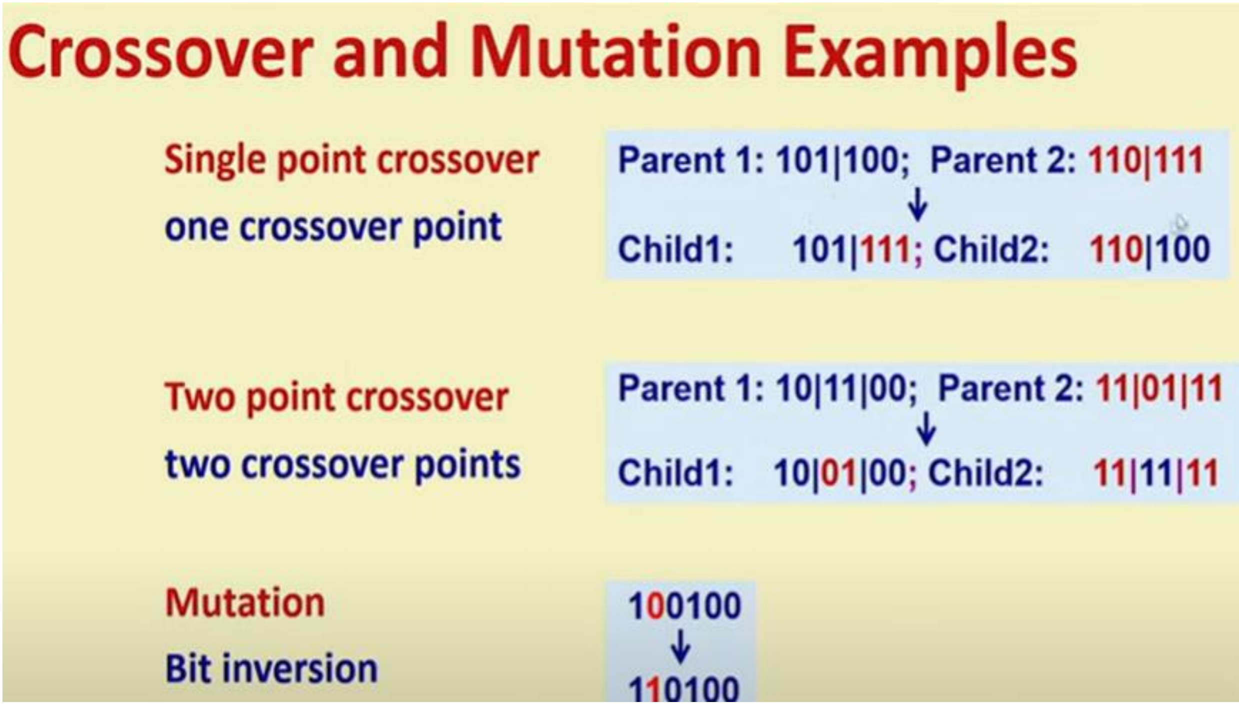
1. Binary Encoding
2. Real Encoding
3. Permutation Encoding
4. Value Encoding
5. Tree Encoding

Which one to use ? When?

Selection Schemes

1. Roulette wheel selection without scaling
2. Roulette wheel selection with scaling
3. Stochastic tournament selection with a tournament size of two
4. Remainder stochastic sampling without replacement
5. Remainder stochastic sampling with replacement
6. Elitism

Which one to use? When?



Parameters for GA

1. Population size - population size is problem specific. A good population size is about 20-30, however even sizes of 50-100 are reported.
2. The best population size depends on the size of encoded string (chromosomes). More the encoded sizes, more should be the population size of.
3. Crossover probability - should be high generally, about 80%-95%. (for some problems it could even be as low as 60%.)
4. Mutation rate should be very low. Best rates seems to be about 0.5%-1%.
5. Crossover and mutation type: Operators depend on the chosen encoding and on the problem.

Benefits of Genetic Algorithms

1. Easy to understand and modular in structure - separate from application
2. Supports multi-objective optimization
3. Good for "noisy" environments
4. Solution is obtained all the time - solution quality improves with additional knowledge gained
5. Inherently parallel; easily distributed
6. Easy to exploit previous or alternate solutions
7. Flexible building blocks for hybrid applications

When to Use Genetic Algorithm

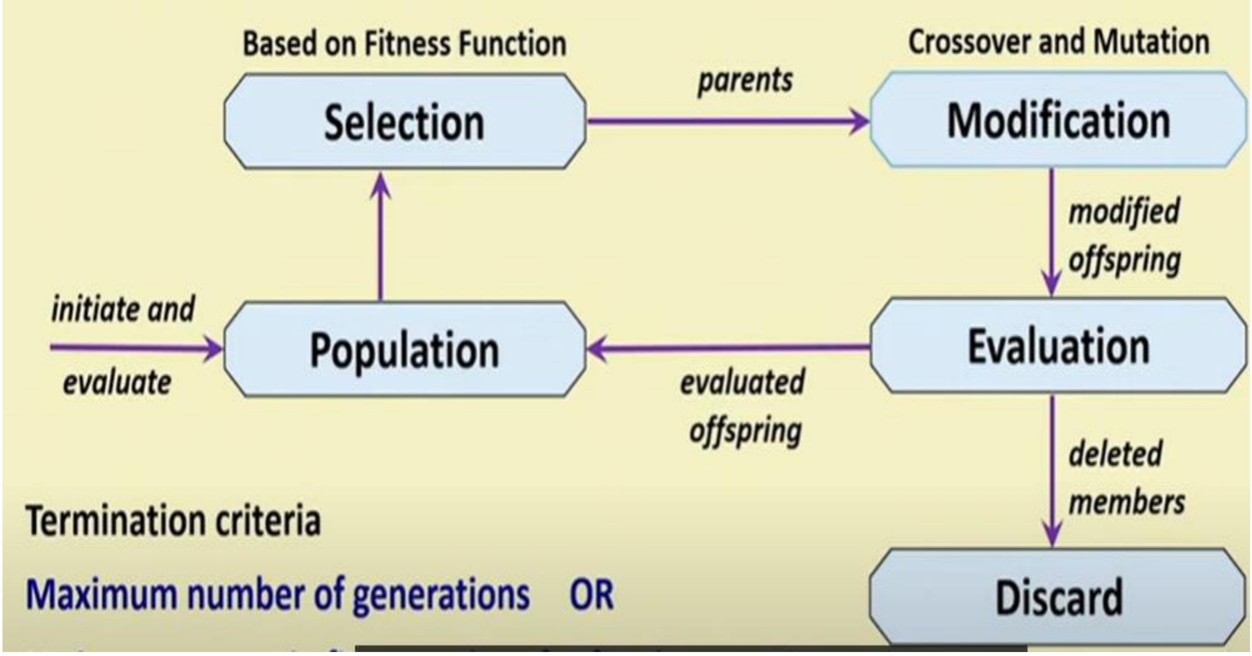
(i) Alternate solutions are too slow or overly complicated
(ii) Need an exploratory tool to examine new approaches

(iii) Problem is similar to one that has already been successfully solved by using GA
(iv) Want to hybridize with an existing solution
(v) Benefits of the GA technology meet key problem requirements
    (A) Near-optimal solution will suffice
    (B) Adequate computational power is available
    (C) The problem does converge to an optimal solution

Some GA Application Types

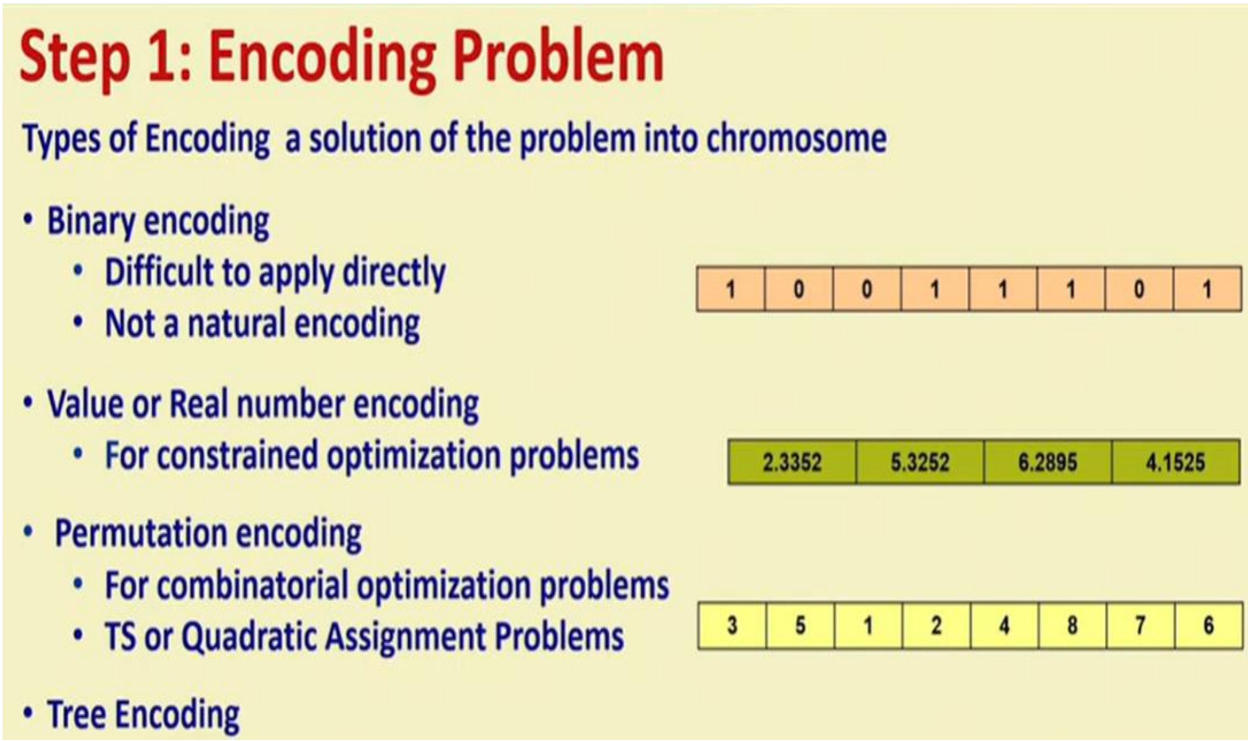
Domain	Application Types
Control	gas pipeline, pole balancing, missile evasion, pursuit
Design	semiconductor layout, aircraft design, keyboard configuration, communication networks
Scheduling	manufacturing, facility scheduling, resource allocation
Robotics	trajectory planning
Machine Learning	designing neural networks, improving classification algorithms, classifier systems
Signal Processing	filter design
Game Playing	poker, checkers, prisoner's dilemma
Combinatorial Optimization	set covering, travelling salesman, routing, bin packing, graph colouring and partitioning

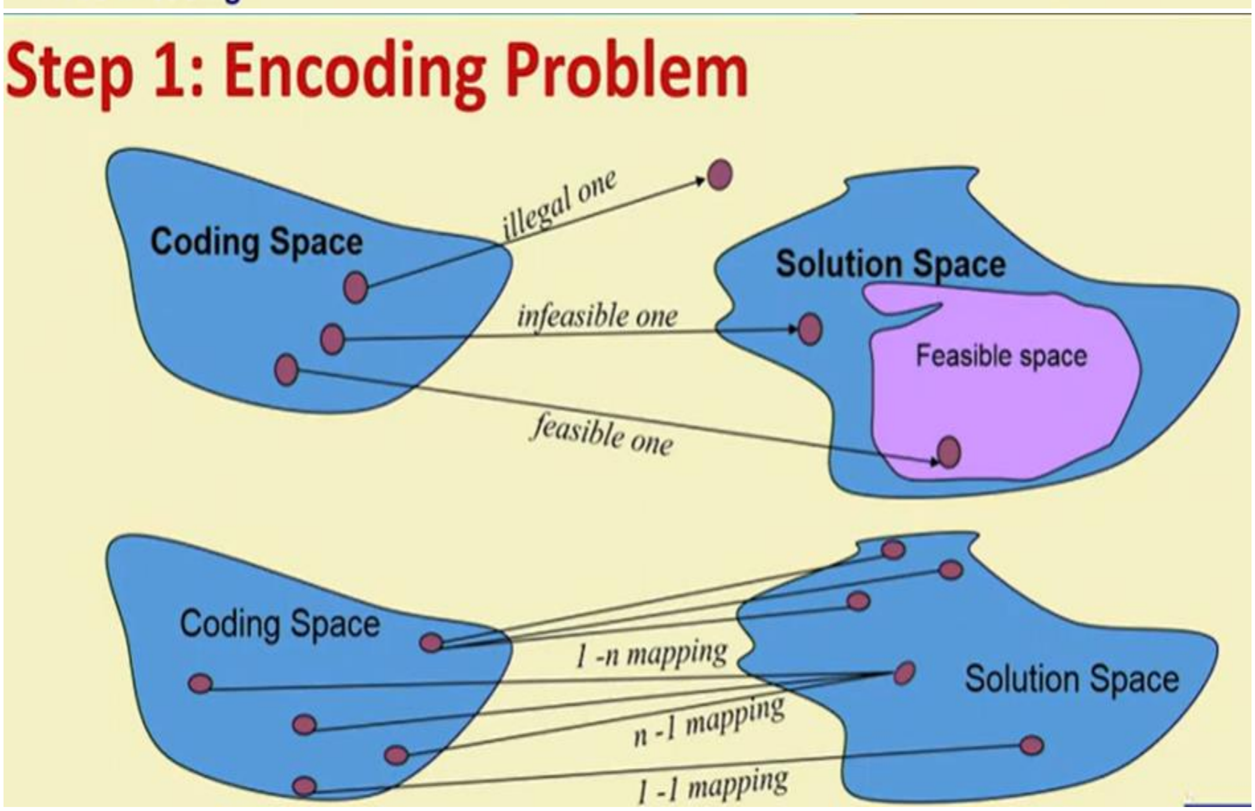
GA: Evolutionary Cycle



Termination Criteria:-

- Maximum number of generations OR No improvement in fitness values for fixed generation





Step 1: Encoding a Problem

Critical issues with Encoding

- **Feasibility** of a chromosome
  - solution decoded from a chromosome lies in a feasible region of the problem
- **Legality** of a chromosome
  - chromosomes represent a solution to a problem
- **Uniqueness** of mapping (Between Chromosomes and solution to the problem)
  – Between one to many, many to one, and one to one, **one to one mapping** is highly desirable with one chromosome representing only one solution to the problem.

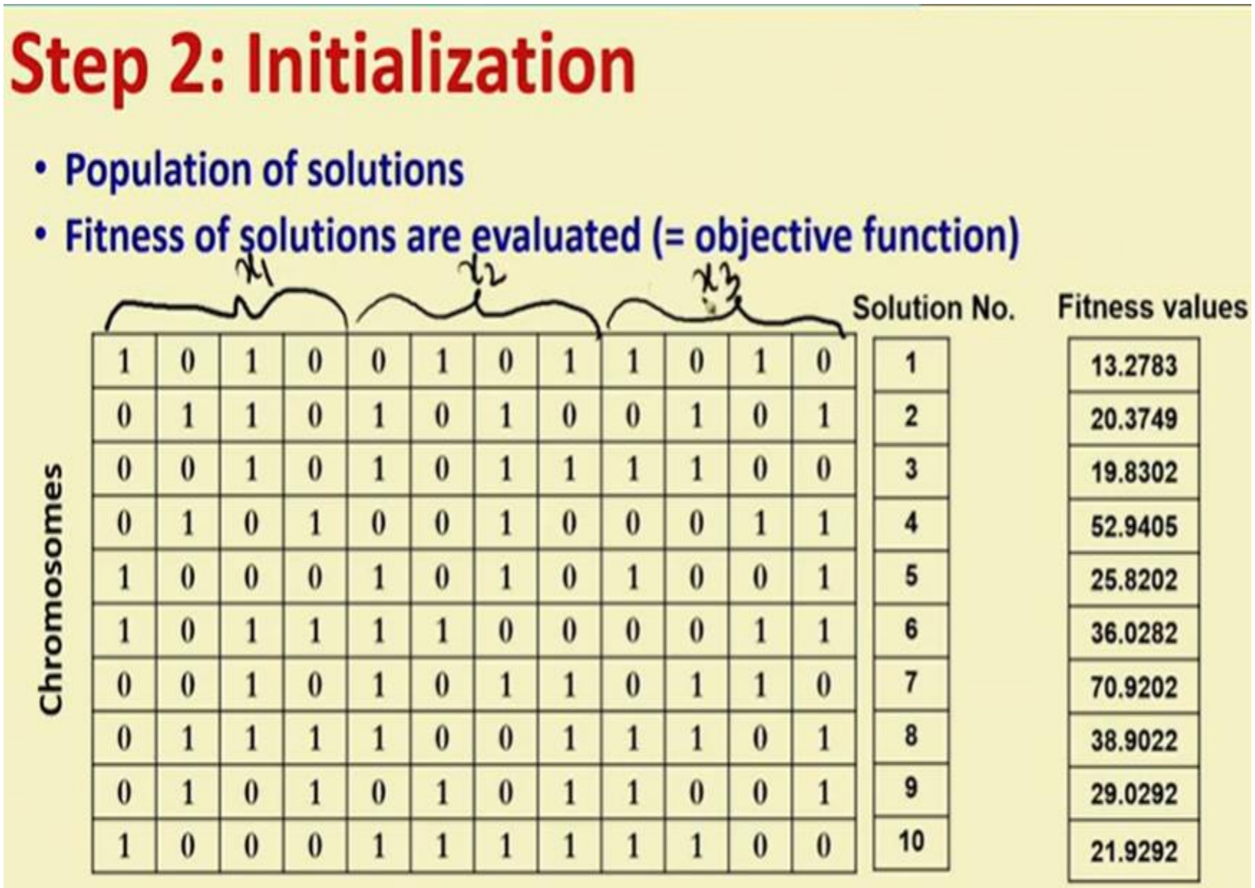
Step 2: Initialization

Create initial population of solutions

- Through a process of randomization
- Through a Local search procedure
- Selecting only Feasible Solutions

Consider a Problem: Minimize: F (x1, x2, x3)
With Binary encoding, we have:

$$\underbrace{\boxed{1}\boxed{0}\boxed{1}\boxed{1}}_{x_1}\underbrace{\boxed{0}\boxed{0}\boxed{1}\boxed{1}}_{x_2}\underbrace{\boxed{1}\boxed{0}\boxed{0}\boxed{1}}_{x_3}$$



Step 2: Initialization

- Population of solutions
- Fitness of solutions are evaluated (= objective function)

Chromosomes

x_1				x_2				x_3				Solution No.	Fitness values
1	0	1	0	0	1	0	1	1	0	1	0	1	13.2783
0	1	1	0	1	0	1	0	0	1	0	1	2	20.3749
0	0	1	0	1	0	1	1	1	1	0	0	3	19.8302
0	1	0	1	0	0	1	0	0	0	1	1	4	52.9405
1	0	0	0	1	0	1	0	1	0	0	1	5	25.8202
1	0	1	1	1	1	0	0	0	0	1	1	6	36.0282
0	0	1	0	1	0	1	1	0	1	1	0	7	70.9202
0	1	1	1	1	0	0	1	1	1	0	1	8	38.9022
0	1	0	1	0	1	0	1	1	0	0	1	9	29.0292
1	0	0	0	1	1	1	1	1	1	0	0	10	21.9292

Step 3: Selection ("Survival of the fittest")

Sampling Mechanisms: select chromosomes from sampling space.

1. **Stochastic Sampling**
2. **Roulette Wheel selection:**
   (a) Determine survival probability proportional to fitness value.
   (b) randomly generate number between [0, 1] and select the individual.
3. **Deterministic Samplings**
   select best individuals from the parents and offspring with no duplication of the individuals
4. **Mixed Samplings**

   both random and deterministic sampling.

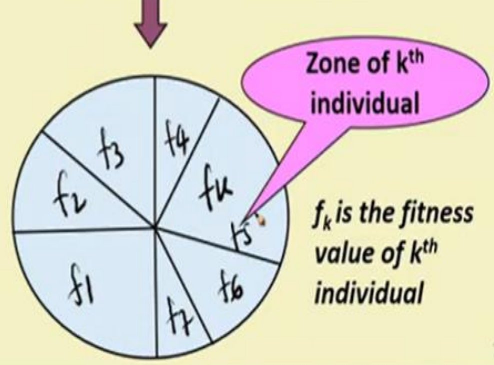
Step 3: Selection ("Survival of the fittest")

Sampling Mechanisms: select chromosomes from sampling space

- Stochastic Sampling
- Roulette Wheel selection:
  - Determine survival probability proportional to fitness value
  - randomly generate number between [0, 1] and select the individual
- Deterministic Samplings
  - select best individuals from the parents and offspring with no duplication of the individuals
- Mixed Samplings
  - both random and deterministic sampling

Selection probability for k^{th} individual $$p_k = \frac{f_k}{\sum_{j=1}^{pop\_size} f_j}$$

Calculate cumulative probability and construct roulette wheel based on p_k



Selection Schemes

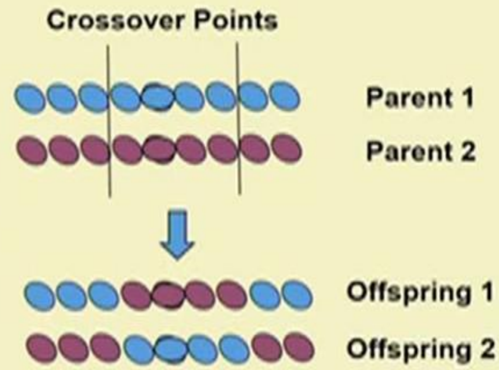
- Roulette wheel selection without scaling
- Roulette wheel selection with scaling
- Stochastic tournament selection with a tournament size of two
- Remainder stochastic sampling without replacement
- Remainder stochastic sampling with replacement
- Elitism

<u>Which one to use?</u>

<u>When?</u>

Step 4: Reproduction

- Crossover operation (Based on crossover probability)
- Select parents from population based on crossover probability
- Randomly select two points between strings to perform crossover operation
- Perform crossover operations on selected strings
- Known for Local search operation



Step 4: Reproduction

- For the example of optimization problem
  - Let the crossover probability be 0.8

Solution No.	Chromosomes												Random values [0,1]		OK for Crossover?
1	1	0	1	0	0	1	0	1	1	0	1	0	0.9502	> 0.8	NO
2	0	1	1	0	1	0	1	0	0	1	0	1	0.2191	< 0.8	YES
3	0	0	1	0	1	0	1	1	1	1	0	0	0.4607	< 0.8	YES
4	0	1	0	1	0	0	1	0	0	0	1	1	0.6081	< 0.8	YES
5	1	0	0	0	1	0	1	0	1	0	0	1	0.8128	> 0.8	NO
6	1	0	1	1	1	1	0	0	0	0	1	1	0.9256	> 0.8	NO
7	0	0	1	0	1	0	1	1	0	1	1	0	0.7779	< 0.8	YES
8	0	1	1	1	1	0	0	1	1	1	0	1	0.4596	< 0.8	YES
9	0	1	0	1	0	1	0	1	1	0	0	1	0.9817	> 0.8	NO
10	1	0	0	0	1	1	1	1	1	1	0	0	0.7784	< 0.8	YES

Step 4: Reproduction

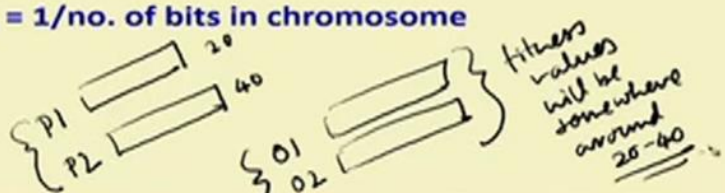
Solution Selected	Parents Selected	Offspring
2	0 1 1 | 0 1 0 1 0 | 0 1 0 1	0 1 1 0 1 0 1 1 0 1 0 1
3	0 0 1 | 0 1 0 1 1 | 1 1 0 0	0 0 1 0 1 0 1 0 1 1 0 0
4	0 1 0 1 0 0 | 1 0 0 0 | 1 1	0 1 0 1 0 0 1 1 0 1 1 1
7	0 0 1 0 1 0 | 1 1 0 1 | 1 0	0 0 1 0 1 0 1 0 0 0 1 0
8	0 1 | 1 1 1 | 0 0 1 1 1 0 1	0 1 0 0 1 0 0 1 1 1 0 1
10	1 0 | 0 0 1 | 1 1 1 1 1 0 0	1 0 1 1 1 1 1 1 1 1 0 0

Crossover Points

Step 4: Reproduction

Mutation operation (based on mutation probability p_m)

- each bit of every individual is modified with probability pm
- main operator for global search (looking at new areas of the search space)
- p_m usually small {0.001,...,0.01}
- rule of thumb p_m = 1/no. of bits in chromosome



Step 4: Reproduction

For optimization problem

Minimize: F (x_1, x_2, x_3)

- Let p_m = 1/12 = 0.083
- Generate Random number [0,1] for each bit
- Select bits having probability less than p_m
- Interchange the bits with each other

i^{th} solution string from the population

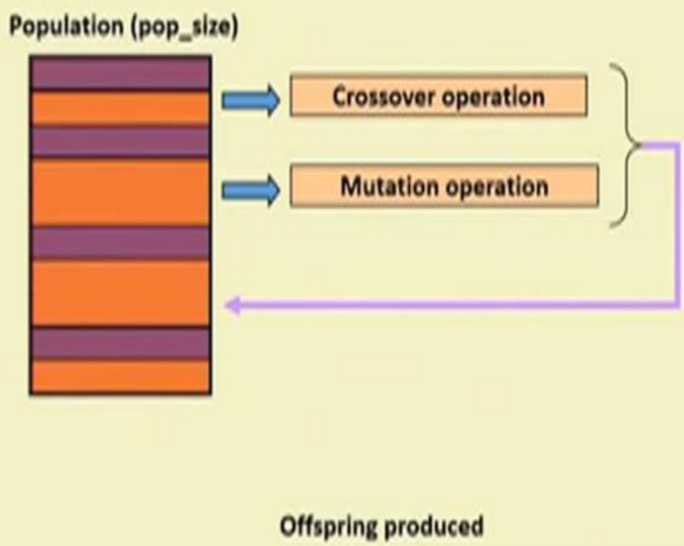
0	0	1	0	1	0	1	1	0	1	0	0
0.12	0.57	0.62	0.31	0. 01	0.73	0.83	0.63	0.02	0.26	0.94	0.63

Mutation

0	0	1	0	0	0	1	1	1	1	0	0

Step 5: Generating Offspring

- Directs the search towards promising regions in the search space
- Basic issues involved in selection phase:
- Sampling space: Parents and Offspring
- Regular sampling space: all offspring + few parent = pop_size
- Enlarged sampling space: All offspring + All parent



Summary of Genetic Algorithms

```
Begin
{
    initialize population;
    evaluate population;
    while (Termination Criteria Not Satisfied)
    {
        select parents for reproduction;
        perform Crossover and mutation;
        evaluate population;
    }
}
```

Parameters for GA

- Empirical studies of GAs show the following:
- Crossover rate should be high generally, about 80%-95%. (for some problems it could even be as low as 60%.)
- Mutation rate should be very low. Best rates seems to be about 0.5%-1%.
- Crossover and mutation type: Operators depend on the chosen encoding and on the problem.

Parameters for GA

- Population size - very big population size usually does not improve performance of GA – speed actually reduces. Good population size is about 20-30, however even sizes of 50-100 are reported as the best.
- The best population size depends on the size of encoded string (chromosomes). More the encoded sizes, more should be the size of population.

Parameters for GA

- Selection: Basic roulette wheel selection can be used, but sometimes rank selection can be better. Elitism should be used for sure if you do not use other method for saving the best found solution. You can also try steady state selection.
- Encoding: Encoding depends on the problem and also on the size of instance of the problem.

Travelling Salesman Problem

A salesperson from City A must visit every month 8 other cities B, C, D, E exactly once and come back to A. Find his minimum distance tour using Genetic Algorithm given the following distance matrix:

	City 1	City 2	City 3	City 4	City 5	City 6	City 7	City 8	City 9
City 1	0	12	22	15	21	21	15	21	21
City 2		0	8	11	13	13	11	13	13
City 3			0	16	18	18	16	18	18
City 4				0	19	19	16	19	19
City 5					0	14	19	20	20
City 6						0	12	12	21
City 7							0	22	13
City 8								0	18
City 9									0

TS Problem: Encoding

- Which encoding should be used?
- Permutation encoding

  Common in Travelling Salesman problems or Task Ordering problems.
- Here each chromosome is a number string representing a position in a sequence.
- Chromosome A 1 5 3 2 6 4 7 9 8
- Chromosome B 8 5 6 7 2 3 1 4 9
- What should be the fitness function?

856723149

149856723

1-4-9-8-5-6-7-2-3

TS Problem: Fitness Function

	City 1	City 2	City 3	City 4	City 5	City 6	City 7	City 8	City 9
City 1	0	12	22	15	21	21	15	21	21
City 2		0	8/8	11	13	13	11	13	13
City 3			0	16	18	18	16	18	18
City 4				0	19	19	16	19	19
City 5					0	14	19	20	20
City 6						0	12	12	21
City 7							0	22	13
City 8								0	18/18
City 9									0

Fitness Function = Maximize F = 1/Distance

Chromosome	Route	Total Distance travelled (D)	Fitness Function (F = 1/D)
1 5 3 2 6 4 7 9 8	1-5-3-2-6-4-7-9-8-1	21+18+8+13+19+16+13+18+21 = 147	F = 1/147 = 0.0068927
8 5 6 7 2 3 1 4 9	1-4-9-8-5-6-7-2-3-1	15+19+18+20+14+12+11+8+22 = 139	F = 1/139 [illegible]

TS Problem: Crossover and Mutation

Single Point Crossover

- one crossover point
- Copy first parent till crossover point.
- add number not present in offspring yet in sequence as present in second parent.

	Parent 1:	6 2 3 4 5 | 1 7 8 9	Parent 2:	4 5 3 6 8 | 9 7 2 1
=>>	Child1:	6 2 3 4 5 | 8 9 7 1	Child 2:	4 5 3 6 8 | 2 1 7 9

Order Changing Mutation

6 2 3 4 5 8 9 7 1 => 6 2 8 4 5 3 9 7 1

TS Problem: Other Considerations

- Which selection scheme to use?

  Roulette Wheel, Tournament, or Remainder Stochastic?

- Should Elitism be used?
- What should be the crossover and mutation probability?
- Are trade-offs achieved between Selection pressure and Population diversity?
- Convergence of the solution? – Stuck in local mimima?